

План занятия

Цель занятия

Показать практическую работу PostgreSQL/EDB в составе высокодоступного кластера и разобрать на стенде, как проявляются:

- транзакционность;
- уровни изоляции;
- консистентность данных;
- блокировки;
- масштабируемость;
- высокая доступность;
- CAP и PACELC.

Основной акцент занятия — не на теоретических определениях, а на том, как эти свойства проявляются в реальной работе кластера.

Используемый стенд

Для занятия используется заранее подготовленный HA-кластер PostgreSQL/EDB:

- 3 узла PostgreSQL/EDB под управлением Patroni;
- 3 узла etcd;
- HAProxy для маршрутизации read/write-трафика;
- PgBouncer для пула подключений;
- тестовая база данных;
- тестовые таблицы для транзакций, блокировок, нагрузки и failover;
- общий endpoint для подключения клиента;
- возможность остановки primary и узлов etcd.

Роли компонентов:

- PostgreSQL/EDB хранит данные и выполняет SQL-запросы;
- Patroni управляет ролями узлов и выполняет failover;
- etcd используется Patroni для хранения состояния кластера и выбора leader;
- HAProxy направляет запросы на актуальный primary или replicas;
- PgBouncer снижает нагрузку от большого количества клиентских подключений.

Ход занятия

1. Проверка состояния кластера

Сначала показывается текущее состояние кластера:

```
patronictl list
```

Проверяется:

- какой узел сейчас является primary;
- какие узлы работают как replicas;
- доступны ли все узлы;
- есть ли replication lag.

Далее проверяется маршрутизация через endpoints.

Для write-endpoint:

```
SELECT pg_is_in_recovery();
```

Ожидаемый результат:

```
false
```

Для read-endpoint:

```
SELECT pg_is_in_recovery();
```

Ожидаемый результат:

```
true
```

Вывод: приложение подключается не к конкретному серверу PostgreSQL/EDB, а к стабильной точке входа. Распределение запросов выполняется через HAProxy и PgBouncer.

2. Транзакции и уровни изоляции

Демонстрации выполняются на текущем primary в двух параллельных сессиях.

READ COMMITTED

В первой сессии открывается транзакция и читается строка.

Во второй сессии эта же строка изменяется и фиксируется через `COMMIT`.

После этого первая сессия повторно читает строку.

Ожидаемый результат: повторный `SELECT` видит новое зафиксированное значение.

Вывод: `READ COMMITTED` не допускает dirty read, но каждый запрос внутри транзакции получает новый committed snapshot.

REPEATABLE READ

Повторяется тот же сценарий, но первая транзакция запускается с уровнем изоляции `REPEATABLE READ`.

Ожидаемый результат: первая транзакция продолжает видеть старое значение, даже если во второй сессии изменение уже зафиксировано.

Вывод: `REPEATABLE READ` фиксирует снимок данных на момент начала транзакции.

SERIALIZABLE

Демонстрируется сценарий с бизнес-ограничением: хотя бы один врач должен остаться дежурным.

Две параллельные транзакции:

- проверяют количество дежурных врачей;
- снимают с дежурства разных врачей;
- пытаются зафиксировать изменения.

Ожидаемый результат: на уровне `SERIALIZABLE` одна из транзакций завершается ошибкой сериализации.

Вывод: `SERIALIZABLE` обеспечивает наиболее строгую корректность конкурентного выполнения, но требует retry-логики на стороне приложения.

3. Блокировки и консистентность

В первой сессии открывается транзакция и выполняется `UPDATE` строки без `COMMIT`.

Во второй сессии выполняется попытка изменить ту же строку.

Ожидаемый результат: вторая сессия ожидает освобождения блокировки.

Состояние ожидания проверяется через `pg_stat_activity`:

```
SELECT
    pid,
    state,
    wait_event_type,
    wait_event,
    query
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY pid;
```

Далее показывается работа ограничений и `ROLLBACK`:

- попытка записать некорректное значение, например отрицательный баланс, завершается ошибкой `CHECK`;
- если внутри транзакции одна операция прошла успешно, а следующая завершилась ошибкой, после `ROLLBACK` первая операция тоже не сохраняется.

Вывод: консистентность обеспечивается не одним механизмом, а сочетанием транзакций, блокировок, ограничений, уровней изоляции и корректной логики приложения.

4. Масштабируемость и нагрузка

Сначала показывается PgBouncer:

```
SHOW POOLS;
```

Объясняется разница между прямыми подключениями к PostgreSQL/EDB и подключениями через пул соединений.

Вывод: PgBouncer уменьшает нагрузку от большого количества клиентских подключений и помогает базе стабильнее работать при высокой конкуренции.

Затем выполняются короткие нагрузочные тесты через `pgbench`:

```
pgbench -c 1 -T 20
pgbench -c 20 -T 20
```

```
pgbench -c 80 -T 20
```

Сравниваются:

- TPS;
- average latency;
- ошибки или просадки производительности.

Отдельно демонстрируется hot row contention:

1. все клиенты обновляют одну и ту же строку;
2. клиенты обновляют разные случайные строки.

Ожидаемый результат: при обновлении одной строки производительность хуже из-за блокировок и очереди транзакций.

Вывод: масштабируемость зависит не только от количества узлов и подключений, но и от модели данных. Одна часто изменяемая строка может стать узким местом всей системы.

5. Высокая доступность и failover

Проверяется нормальная работа кластера:

- есть текущий primary;
- replicas доступны;
- HAProxy направляет write-запросы на primary;
- приложение успешно пишет данные через общий endpoint.

Запускается тестовый клиент, который постоянно выполняет запись:

```
while true; do
  psql "$DATABASE_URL" -c "INSERT INTO ha_demo(value) VALUES ('tick ' ||
now());"
  sleep 1
done
```

Параллельно отслеживается состояние кластера:

```
watch -n 1 patronictl list
```

После этого останавливается текущий primary:

```
docker stop pg-node-1
```

или:

```
systemctl stop patroni
```

Проверяется:

- старый primary становится недоступен;
- Patroni выбирает новый leader;
- одна из replicas становится primary;
- HAProxy начинает направлять write-трафик на новый primary;
- клиент после короткого разрыва продолжает запись через тот же endpoint.

Проверяется новый primary:

```
SELECT pg_is_in_recovery();
```

Ожидаемый результат:

```
false
```

Проверяются последние записи:

```
SELECT * FROM ha_demo ORDER BY id DESC LIMIT 5;
```

Вывод: высокая доступность означает, что при отказе primary кластер автоматически выбирает новый primary, а приложение продолжает работать через тот же endpoint. Кратковременный обрыв соединений во время failover является нормальным поведением, поэтому приложение должно уметь повторять операции.

6. CAP на стенде

CAP показывается на реальном поведении кластера при отказах.

Отказ primary

На фоне постоянной записи через общий endpoint останавливается текущий primary.

Ожидаемое поведение:

- primary становится недоступен;
- Patroni выбирает новый primary;
- HAProxy перенаправляет write-трафик;
- клиент после короткой паузы продолжает запись.

Вывод: кластер старается сохранить доступность сервиса за счёт автоматического failover. При асинхронной репликации возможен риск потери последних подтверждённых транзакций, если они не успели попасть на replica до отказа primary.

Потеря quorum в etcd

Останавливаются два узла etcd из трёх:

```
docker stop etcd-2 etcd-3
```

После этого проверяется состояние кластера:

```
patronictl list
```

Также проверяется возможность записи через write-endpoint:

```
INSERT INTO ha_demo(value) VALUES ('after etcd quorum loss');
```

Фиксируется фактическое поведение стенда: запись проходит, блокируется или становится недоступной после потери leader lock.

Вывод: без quorum в etcd кластер не может надёжно принимать решения о лидерстве. Чтобы не допустить split-brain и появления двух primary, система ограничивает доступность записи. Это практическое проявление выбора consistency вместо availability.

Дополнительно сравнивается асинхронная и синхронная репликация:

- async replication даёт меньшую задержку и более быстрый failover, но допускает риск потери последних изменений;
- sync replication снижает риск потери данных, но увеличивает latency и может остановить запись при проблемах с синхронной replica.

7. PACELC на стенде

PACELC используется для объяснения компромиссов не только во время отказов, но и в штатной работе системы.

Формула:

```
If Partition: Availability or Consistency  
Else: Latency or Consistency
```

На стенде это проявляется так:

- при отказе или потере quorum кластер выбирает между доступностью и консистентностью;
- в штатном режиме async replication снижает latency, но ослабляет гарантию свежести данных;
- sync replication усиливает consistency/durability, но увеличивает задержку commit;
- чтение с replicas разгружает primary, но может вернуть не самые свежие данные;
- чтение с primary даёт более актуальные данные, но увеличивает нагрузку на основной узел.

Вывод: компромиссы существуют не только при авариях. Даже в нормальном режиме работы кластер постоянно балансирует между задержкой, свежестю данных, нагрузкой и устойчивостью.

Итог занятия

По итогам занятия должно быть показано, что HA-кластер PostgreSQL/EDB повышает доступность базы данных, но не делает систему «автоматически идеальной».

Основные выводы:

- PostgreSQL/EDB обеспечивает транзакции, уровни изоляции, блокировки, ограничения и rollback;
- Patroni управляет ролями узлов и выполняет failover;
- etcd нужен для координации и выбора leader;
- HAProxy направляет запросы на узлы с актуальными ролями;
- PgBouncer снижает нагрузку от большого числа подключений;
- при failover возможен кратковременный разрыв соединений;
- приложение должно уметь повторять операции после ошибок подключения или сериализации;
- масштабируемость зависит не только от инфраструктуры, но и от схемы данных и характера нагрузки;

- CAP и PACELC на практике проявляются через выбор между доступностью, консистентностью и задержкой.

Основной итог: HA-кластер повышает отказоустойчивость PostgreSQL/EDB, но требует понимания его поведения при конкурентных операциях, нагрузке, отказе primary и проблемах с quorum.